

AI MetroFlow

AI Platform for Metro Crowd Management & Scheduling

Large-Scale Synthetic Transportation Dataset for Indian Metro Systems

Attribute	Value
Total records	571,540
Data window	2024-10-01 to 2024-12-29 (90 days)
Metro networks	17
Cities covered	23
Unique stations	725
Datasets (CSV files)	8
Quality checks passed	15/15

Document: dataset description, sources, generation methodology, feature dictionary, ML use cases, quality report, analytics, limitations and roadmap.

1. Dataset Description & Purpose

AI MetroFlow is a relational, ML-ready transportation dataset that emulates the operational data footprint of Indian metro networks **without any CCTV, computer-vision or facial-recognition data**. Every signal is derived from the kinds of records a metro authority already owns: ticketing, smart-card taps, automatic-fare-collection (AFC) entry/exit logs, station statistics, train operations telemetry, schedules and external context (weather / holidays / events).

It is built to train and benchmark models for passenger crowd prediction, station congestion analysis, demand forecasting, train-frequency optimisation, peak-hour prediction and delay-impact analysis, and to power FastAPI backends and React analytics dashboards for smart-city demonstrations.

Intended users

- Metro rail authorities & operations control centres
- Smart-city transportation departments
- Urban mobility operators & planning teams
- Public-transport analytics / data-science teams
- Hackathon and academic project teams

1.1 The eight datasets

File	Grain / one row is	Records
metro_stations.csv	one unique station (interchanges merged)	725
passenger_flow.csv	station x day x operating-hour footfall	224,010
ticket_transactions.csv	a single ticket / smart-card journey	65,000
train_operations.csv	a train stop event (arrival/departure/delay)	44,333
train_occupancy.csv	train load sampled at a station	10,902
external_factors.csv	city x day weather / holiday / event context	2,070
train_schedule.csv	line x time-slot x day-type schedule & recommendation	490
metro_ai_training_data.csv	engineered feature/label row (station-hour)	224,010

Total: 571,540 records. The two prediction tables (passenger_flow and metro_ai_training_data) each exceed 200,000 rows, giving deep hourly time-series depth for LSTM / gradient-boosting models.

2. Network & Geographic Coverage

Station names and line assignments are **real**, compiled from public metro network maps. The following networks are represented:

Metro network	Stations
Delhi Metro	222
Bengaluru Metro	68
Mumbai Metro	67
Hyderabad Metro	57
Chennai Metro	37
Kolkata Metro	37
Nagpur Metro	35
Ahmedabad Metro	33
Pune Metro	29
Kochi Metro	25
Lucknow Metro	21
Kanpur Metro	21
Noida Metro	20
Bhopal Metro	15
Indore Metro	15
Jaipur Metro	12
Agra Metro	11

2.1 Stations by state

State	Stations
Delhi NCR	222
Maharashtra	131
Uttar Pradesh	73
Karnataka	68
Telangana	57
Tamil Nadu	37
West Bengal	37
Gujarat	33
Madhya Pradesh	30
Kerala	25
Rajasthan	12

2.2 Stations by category

Category	Stations
Residential	264
Commercial	203

IT Hub	88
Educational Zone	79
Industrial Area	54
Railway Connection	26
Airport	11

Interchange (multi-line) stations: **52**.

3. Data Sources

- **Station master (real):** station names, line memberships, terminals and opening years compiled from publicly available metro network maps of DMRC, MMRDA / Maha-Metro, BMRCL, HMRL, CMRL, Kolkata Metro, KMRL, GMRC, UPMRC, MEGA/JMRC, NMRC and others.
- **Geo-coordinates (approximated):** each station is placed by linear interpolation along its line's real terminal-to-terminal corridor with small Gaussian jitter, so stations are geographically ordered and city-plausible. Coordinates are approximate and NOT survey-grade.
- **Operational & behavioural signals (synthetic):** footfall, entry/exit splits, occupancy, delays, fares and demand are generated from a rules-based stochastic model calibrated to well-known Indian metro behaviour (see Section 4).
- **External context (synthetic, calendar-aligned):** public holidays and festivals use the real 2024 calendar for the data window; weather and events are sampled from season/city-appropriate distributions.

4. Data Generation Methodology

The generator is fully deterministic (fixed random seed = 42) and reproducible.

4.1 Demand model

Each station has a **daily average footfall** derived from a city-tier multiplier x category weight x interchange bonus x network-age factor x noise. Hourly volume is obtained by multiplying daily footfall by a category-specific hourly shape profile, then applying multiplicative factors:

- **Hour-of-day profile** per category (commercial/IT peak at 09:00 & 18:00; residential peaks morning-out / evening-in; airport flat; educational mid-day).
- **Entry/exit split** flips with direction of commute (residential is entry-heavy in the morning, commercial is exit-heavy).
- **Weekend factor** suppresses IT / office / education stations far more than commercial or residential ones.
- **External factor** = $1 + \text{impact_percentage}/100$ from `external_factors` (holidays reduce office travel, festivals lift commercial/temple stations).
- **Event venue boost** raises 1-3 nearby stations on event days, concentrated in the evening hours.
- **Weather** dampens ridership on rain / heavy-rain / fog days.

4.2 Cross-dataset consistency

`external_factors` is generated first, per city-day; `passenger_flow`, and hence `metro_ai_training_data`, JOIN weather / holiday / festival / event from it. Ticket fares are computed from the real haversine distance between the chosen entry and exit stations using Delhi-Metro-style distance slabs, with a 10% smart-card / pass discount.

5. Feature Dictionary (key columns)

metro_stations.csv

Column	Meaning
station_id	unique PK, e.g. DEL001
metro_name / state / city	network and location
station_name / station_code	real name; short code
line_name	line(s); ' / '-joined for interchanges
latitude / longitude	approximate corridor-interpolated geo-point
opening_year	year the (earliest) line opened
interchange_station	Yes if station serves >1 line
platform_count	number of platforms
daily_average_footfall	modelled daily throughput (entries+exits)
station_category	Commercial / Residential / Airport / Railway Connection / Educational Zone / IT Hub / Industrial Area

passenger_flow.csv

Column	Meaning
record_id	unique PK
date / time	day and hour (05:00-23:00)
station_id	FK to master
entry_count / exit_count / total_passengers	throughput; entry+exit=total
peak_hour_flag	1 if 08-10 or 18-20
weekday / holiday_flag	day name; 1 on public holiday
weather_condition / event_near_station	joined from external_factors
crowd_density_level	Low / Medium / High / Critical

ticket_transactions.csv

Column	Meaning
transaction_id	unique PK
ticket_type	Smart Card / QR Ticket / Token / Monthly Pass
passenger_category	General / Student / Senior Citizen / Women / Differently-abled / Tourist
entry_station / exit_station	journey OD pair (same network)
travel_duration	minutes (distance/speed + access)
fare_amount	INR from distance slab
payment_method	UPI / NCMC / Cash / Metro App / Card

train_operations.csv / train_occupancy.csv

Column	Meaning
train_id	service identifier
arrival/departure/scheduled/actual_time	stop timing
delay_minutes	accumulated delay at stop
train_frequency	headway in minutes
occupancy_percentage	load vs capacity (may exceed 100 at crush)
service_status	Running / Delayed / Cancelled / Maintenance
coach_capacity / current_passengers	occupancy table only

metro_ai_training_data.csv (ML-ready)

Column	Meaning
INPUTS	station_id, hour, day_of_week, previous_hour_passengers, previous_day_average, weather, holiday, e
future_crowd_level	TARGET (classification): next-hour density
future_passenger_count	TARGET (regression): next-hour volume
recommended_train_frequency	TARGET: suggested headway
congestion_probability	TARGET: P(High/Critical) next hour

6. Machine-Learning Use Cases

Use case	Type	Suggested models	Table / target
Next-hour crowd prediction	Classification	RandomForest, XGBoost, LSTM	metro_ai_training_data -> future_crowd_level
Passenger demand forecasting	Regression / TS	XGBoost, LSTM, Prophet/SARIMA	future_passenger_count
Train frequency optimisation	Regression / policy	GBM, rule + RL	train_schedule -> recommended_frequency
Congestion detection	Prob. classification	LogReg, XGBoost	congestion_probability
Peak-hour prediction	Classification	any	peak_hour_flag
Delay-impact analysis	Regression	GBM	train_operations.delay_minutes

Recommended split: train on the first ~70 days, validate/test on the final ~20 days (temporal hold-out) to avoid look-ahead leakage. All lag features in metro_ai_training_data reference only past values.

7. Data-Quality Report

Check	Result
metro_stations.station_id unique	PASS 725 rows
passenger_flow.record_id unique	PASS 224010 rows
ticket_transactions.transaction_id unique	PASS 65000 rows
passenger_flow.station_id -> master FK	PASS 0 orphans
ticket_transactions.station_id -> master FK	PASS 0 orphans
train_operations.station_id -> master FK	PASS 0 orphans
train_occupancy.station_id -> master FK	PASS 0 orphans
metro_ai_training_data.station_id -> master FK	PASS 0 orphans
passenger_flow entry+exit == total	PASS 0 mismatches
passenger_flow no missing key fields	PASS 0 rows with gaps
station coordinates within India bbox	PASS
train_occupancy pct in [0,150]	PASS
fares within slab [9,60]	PASS
crowd_density_level in allowed set	PASS
external_factors = cities x days	PASS 23 cities x 90 days

All quality checks passed. No duplicate primary keys, full referential integrity, arithmetic consistency (entry+exit=total), no missing key fields, and all values within realistic ranges.

8. Analytics Highlights

8.1 Busiest stations (by modelled daily footfall)

Station	City	Metro	Footfall/day
Central Secretariat	Delhi	Delhi Metro	306,017
New Delhi	Delhi	Delhi Metro	303,868
Mayur Vihar Phase-1	Delhi	Delhi Metro	274,696
Dwarka Sector 21	Delhi	Delhi Metro	273,044
Azadpur	Delhi	Delhi Metro	272,277
Anand Vihar ISBT	Delhi	Delhi Metro	269,077
Sarai Kale Khan Nizamuddin	Delhi	Delhi Metro	266,662
Hauz Khas	Delhi	Delhi Metro	265,682
Welcome	Delhi	Delhi Metro	265,516
Ashok Park Main	Delhi	Delhi Metro	248,133

8.2 City comparison (average daily throughput across monitored stations)

City	Avg daily flow
Delhi	1,697,926
Mumbai	907,555
Bengaluru	748,620
Hyderabad	733,628
Chennai	701,295
Kolkata	643,676
Gurugram	416,465
Kochi	355,684
Ahmedabad	347,387
Jaipur	325,045
Lucknow	306,782
Noida	305,367

8.3 Peak hours & behaviour

- Busiest hours network-wide: 19:00, 9:00, 18:00, 8:00 - a clear bimodal morning (08-09) and evening (18-19) commute pattern.
- Average station-hour flow is 3,997 on weekdays vs 3,017 on weekends (office/IT stations drive the drop).
- Crowd-density mix across all flow records: Low 120,658, Medium 66,091, High 34,496, Critical 2,765.
- Average fare Rs 30.4, average journey 26.5 minutes.
- Train service status: Running 38,965, Delayed 5,308, Cancelled 60; mean delay 4.28 min.

9. Data Limitations

- **Synthetic operational data.** Footfall, occupancy, delays and fares are modelled, not measured; absolute magnitudes are plausible but not official AFC counts.
- **Approximate coordinates.** Geo-points are interpolated along real corridors, not surveyed; do not use for precise routing or distance certification.
- **Monitored-station flow.** Hourly passenger_flow covers the busiest 131 stations (top 6 per city); the full master lists all 725 stations.
- **Fixed 90-day window** (Oct-Dec 2024). Long-term seasonality (monsoon vs summer, multi-year growth) is not represented.
- **Simplified fare model** uses Delhi-style distance slabs uniformly across networks; real fares differ per operator.
- **Independent transaction sampling:** ticket ODs are footfall-weighted but not reconciled row-for-row with the aggregate passenger_flow counts.

10. Future Improvements

- Ingest real open-data AFC / GTFS feeds where published to calibrate magnitudes.
- Extend to a full calendar year to capture monsoon and summer seasonality.
- Reconcile ticket_transactions exactly against passenger_flow aggregates.
- Add graph/network features (line topology, betweenness) for GNN models.
- Add real per-operator fare tables and true station coordinates.
- Publish a FastAPI service + React dashboard consuming these CSVs directly.

Generated by the AI MetroFlow synthetic data pipeline - deterministic (seed 42), reproducible via scripts/generate_metroflow.py.